

ICLR 2022 | NeuPL: Fitting a Whole Population of Policies Into One Network

UCL and DeepMind unify self-play, fictitious play, and PSRO in a single conditional network — with convergence guarantees and skills that transfer for free

In strategy games like StarCraft or poker, training a single "best" policy is meaningless: rock always loses to paper, and any lone strategy can be countered. To hold its ground in games like these, an agent needs a whole population of policies that keep each other in check. The catch is how that population is usually grown — one policy at a time. You freeze the existing policies, train a fresh policy from scratch to beat a mixture over them, and repeat.

This recipe is clean in a normal-form game like rock-paper-scissors, where a best response can be solved exactly. But real games are temporal and partially observed, so a best response can only be approximated with expensive deep reinforcement learning. That breaks the loop in two ways. Under a finite budget you can't tell a genuinely converged best response from one stuck on a local plateau, so iterations get truncated on a hand-crafted schedule and the population fills up with under-trained "good responses." And every new policy relearns basic skills — perception, memory, motor control — from scratch, which becomes intractable as opponents grow stronger.

The insight behind NeuPL (Neural Population Learning) is that these policies overlap enormously. They share perception, memory, and motor skills, so training them independently is wasteful. If a single model could hold the entire population and condition its behavior on which opponents it faces, the early skills it learns against weak opponents would directly bootstrap the discovery of exploiters to much stronger ones. This ICLR 2022 paper, from University College London and DeepMind, turns that idea into a unifying framework that keeps its convergence guarantees.

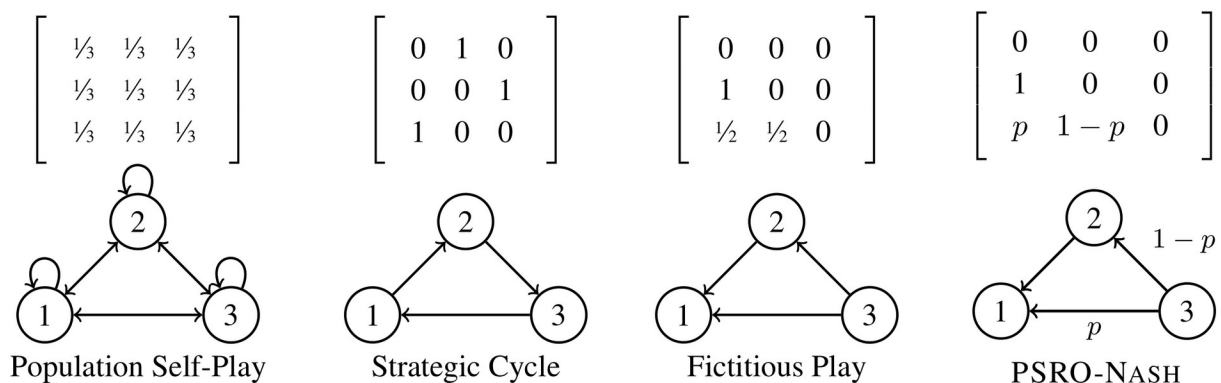


Figure 1. Popular population-learning algorithms written as directed interaction graphs (bottom) or, equivalently, as meta-game mixture-strategy matrices Σ (top). A directed edge from policy i to j with weight σ_{ij} means policy i optimizes against j with probability σ_{ij} . Self-play, a strategic cycle, fictitious play, and PSRO-Nash are all just different choices of Σ within one framework.

Where the old framework gets stuck

Why insist on a population at all? The reason is game-theoretic. In a purely cyclic game like rock-paper-scissors, getting "better" is relative: improving against one opponent means losing to another, so a single point strategy has nowhere to stand. The classic answer, Policy Space Response Oracles (PSRO), grows a population by repeatedly best-responding to a mixture over the existing strategies — but it trains them one at a time and discards the knowledge they could share. Empirical successes like the StarCraft league leaned on close to a thousand agents at enormous cost, which is exactly the case for a far more sample- and skill-efficient alternative.

NeuPL hands two normally hand-tuned knobs — the population size and the per-iteration training budget — back to the algorithm. Instead of a hand-crafted truncation schedule deciding when a policy counts as "done," a meta-graph solver decides the effective population size during training, while every policy shares the same underlying skills.

The method: one network, one interaction graph

Concretely, NeuPL trains a single opponent-conditioned network $\Pi\theta(\cdot|o, \sigma_i)$. Each policy index i is paired with a meta-strategy vector σ_i — a probability distribution that says which other policies it should play against, and how often. The network maximizes a discounted return under a double expectation: first over the sampled opponent σ_j , then over the game dynamics induced by both policies, written $J(\sigma_i) = E_{\{\sigma_j \sim P(\sigma_i)\}} E_{\{a \sim \Pi\theta(\cdot|o, \sigma_i), a' \sim \Pi\theta(\cdot|o', \sigma_j)\}} [\sum_t \gamma^t r_t]$.

To optimize this by reinforcement learning, NeuPL jointly trains an opponent-conditioned action-value function $Q\theta$, and additionally learns a payoff estimator $\phi\omega$ that predicts the expected outcome of any policy pairing directly from the value function — so the full payoff matrix can be evaluated cheaply without replaying every matchup. The interaction graph Σ that assigns opponents can be a fixed matrix, which reproduces fictitious play, or adaptive, recomputed from the learned payoffs U by a meta-graph solver as $\Sigma \leftarrow F(U)$, which reproduces PSRO-Nash. Because the graph, not a hand-crafted schedule, controls which objectives are active, the number of genuinely distinct policies grows on its own as the game demands. When two rows $\sigma_i = \sigma_j$ coincide, their policies collapse into one, so the effective population size emerges automatically.

The paper backs this with a guarantee: for grounded, lower-triangular interaction graphs and a suitable meta-graph solver, NeuPL provably converges to an N -step best response, and with large enough N to a normal-form Nash equilibrium. In other words, it buys the efficiency of sharing and transfer without giving up the game-theoretic convergence that made the iterative methods trustworthy.

Watching it work in rock-paper-scissors

The clearest way to see NeuPL's dynamics is to put it back in rock-paper-scissors, where the whole population can be drawn directly on the strategy simplex. The figure below uses an adaptive interaction graph implementing PSRO-Nash, with an epoch every 10 iterations. Read the three rows separately: the top shows where each strategy sits in the R/P/S triangle, the middle is the learned payoff matrix between strategies, and the bottom is the interaction graph the solver produces as its response to that payoff matrix each epoch.

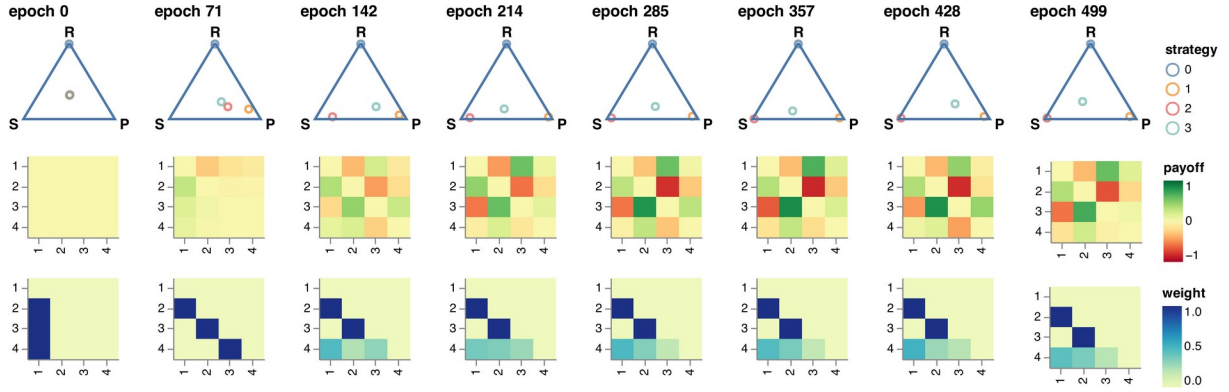


Figure 3. NeuPL in rock-paper-scissors under an adaptive interaction graph implementing PSRO-Nash. (Top) the strategy space explored by the neural population over time; (middle) the learned payoff estimates between strategies; (bottom) the interaction-graph adjacency matrices produced as responses to the payoff matrix. New strategies are pushed toward the corners of the simplex as the payoff matrix and graph expand.

As training proceeds, new strategies are pushed one by one toward the corners of the simplex, a cyclic structure sharpens in the payoff matrix, and the interaction graph expands to match. This is what "let the solver decide who to train against" looks like in the simplest possible game. The real test is whether the same mechanism scales to harder domains.

Scaling to a spatiotemporal, partially observed game

Running-with-scissors lifts rock-paper-scissors into a spatiotemporal, partially observed setting. Players move around a grid world, collect rock, paper, and scissors resources, and see the world only through a narrow 4×4 first-person view, from which they must infer the opponent's hidden inventory. There is no exact best response to solve for here, exactly the regime where PSRO's hand-crafted truncation schedule is most fragile.

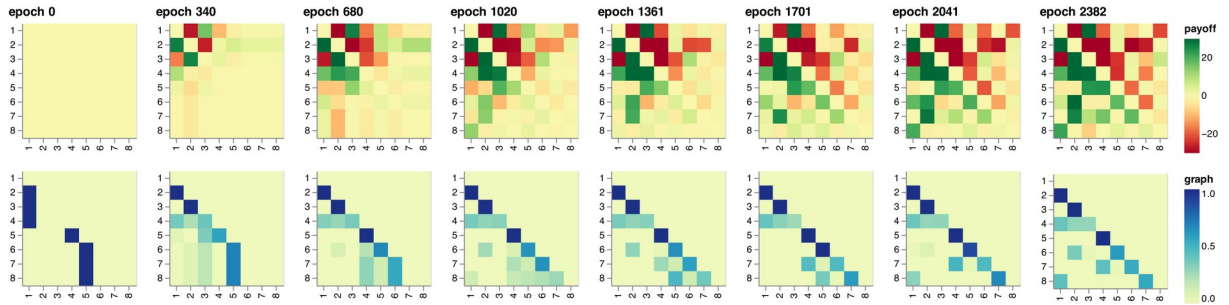


Figure 4. A NeuPL population developing an increasingly sophisticated, diverse set of policies in running-with-scissors. The interaction graph is refreshed every 1,000 gradient updates (an epoch). (Top) the learned payoff estimates U between policies as training progresses; (bottom) the interaction graph $\Sigma \leftarrow \text{FPSRO-N}(U)$ produced in response to the payoff matrix.

The payoff matrix U grows structure out of a blank start and the interaction graph Σ expands alongside it, evidence that the population keeps discovering new mutually-countering strategies — all inside a single network, where the perception and control skills learned early are reused again and again by later policies.

The headline result: more efficient and more robust

The most telling comparison pits a NeuPL population directly against PSRO baselines representing the same set of policies. The figure below only makes sense if you read both axes together: the solid curves are Relative Population Performance (higher means better able to exploit the opponent), and the dashed purple line is the effective population size on the right axis.

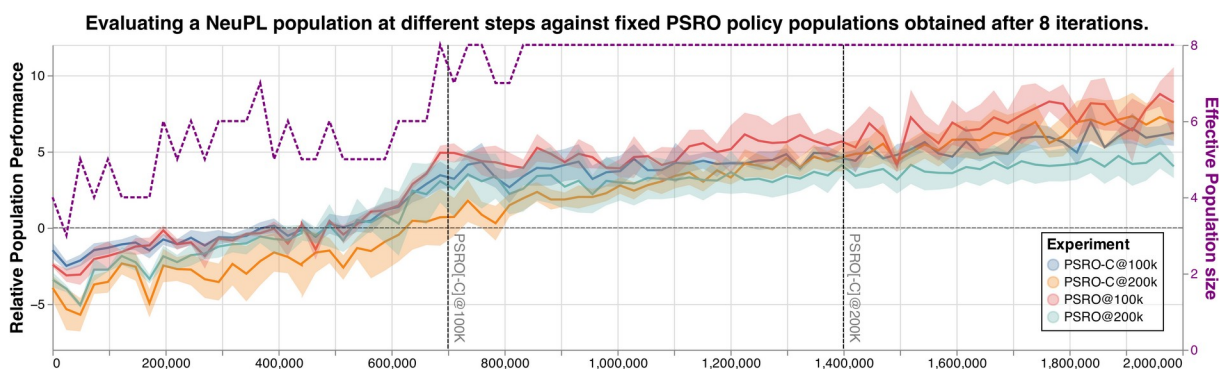


Figure 7. Relative Population Performance between a NeuPL population and the policy populations obtained by PSRO baselines; each PSRO variant is repeated over 3 trials (shaded). The dashed purple line is the effective population size: the rise in relative performance coincides with effective size growing from 5 to 8.

Two things stand out. First, a NeuPL population capped at 8 policies exploits PSRO populations of the same 8 policies — even when each PSRO iteration was given $2\times$ the gradient updates. Second, and more importantly, the gain in relative performance coincides with the effective population size climbing from 5 to 8. That timing suggests the improvement comes from genuinely discovering new strategies rather than overfitting to old ones. Both the continued-training and from-scratch PSRO variants prove equally exploitable, which suggests they fail to build reusable representations.

Why it gets cheaper: the role of transfer

NeuPL's most counterintuitive property is that discovering new strategies gets easier, not harder, as the population expands. A dedicated ablation isolates transfer to test this. A fresh agent is initialized either from scratch or by transferring the encoder and memory components of a NeuPL network trained to epoch 1,000, then asked to beat fixed Nash mixtures of increasing strength.

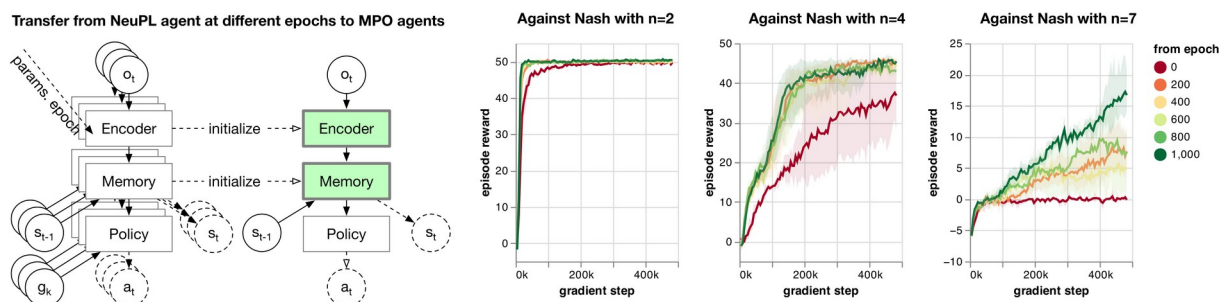


Figure 6. Transferring from a NeuPL agent (left: only the green Encoder and Memory are carried over) into a fresh agent that must learn to exploit fixed Nash mixtures. Red is trained from random initialization; green

transfers from epoch 1,000. Against an easily exploitable $n=2$ mixture the random-init agent catches up; against strong $n=4$ and $n=7$ mixtures it barely learns even with prolonged training, while transfer readily finds the exploit. Each experiment is repeated five times.

Against an easily exploitable two-policy mixture, even the from-scratch agent eventually finds a counter, just more slowly. But against competent mixtures over four or seven policies, the randomly initialized agent barely learns at all despite prolonged training, while the agent that transferred the encoder and memory readily discovers the exploit. This is the paper's central claim made concrete: the larger the population, the thicker the shared skill base, and the easier it becomes to find the next new strategy.

All the way up to MuJoCo Football

Finally, to show the framework survives real scale, the paper applies NeuPL to 2v2 MuJoCo Football — a Game-of-Skills that demands continuous motor control and team coordination at once, with latent strategic cycles underneath.

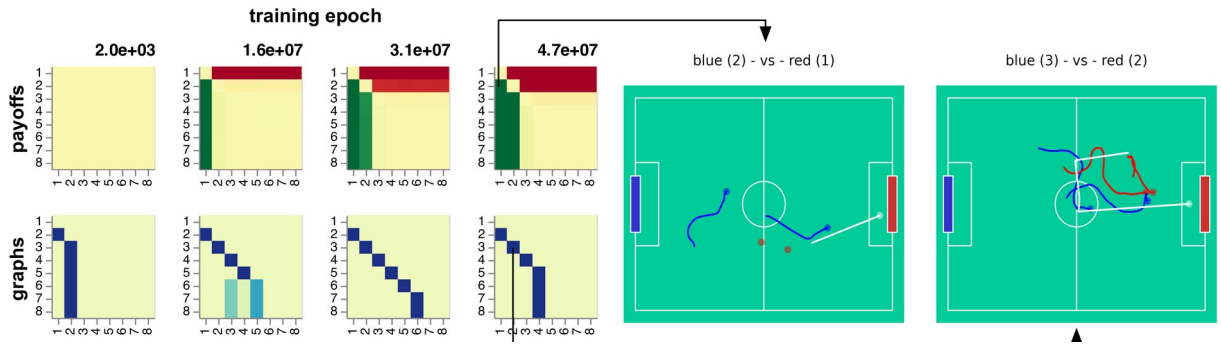


Figure 8. NeuPL implementing FPSRO-N in 2v2 MuJoCo Football. Left: the learned payoff matrices and interaction graphs at successive training epochs. Right: red/blue/white traces are the red players, blue players, and the ball, illustrating the counter relationships that emerge between policies.

Even in this regime, where continuous control and team play are entangled, the payoff matrix grows a clear cyclic structure and the trajectory traces show genuine counter relationships between policies. The same "one network holds the whole population" recipe carries all the way from rock-paper-scissors up to football.

The numbers at a glance

Table 1. NeuPL's core quantitative results.

Metric	Result
NeuPL population size	8, and it exploits an 8-policy PSRO population
PSRO baseline's extra budget	$2\times$ gradient updates (still exploited)
Effective population size growth	$5 \rightarrow 8$ (in step with relative performance)
Effective population size plateau	≈ 12 (across maximum-size caps)
Transfer experiment repeats	5
Nash mixture strengths	$n \in \{2, 4, 7\}$ policies
Evaluation domains	rock-paper-scissors / running-with-scissors / 2v2 MuJoCo Football

Takeaway and honest boundaries

The one-line takeaway: hold the whole population inside a single opponent-conditioned network, let the interaction graph decide who trains against whom, and skills transfer for free across policies. That turns population learning from a sequence of wasteful from-scratch iterations into one transfer-rich training run — cheaper, still convergent, and with the surprising bonus that novel strategies become more accessible, not less, as the neural population grows.

The boundaries are worth stating plainly. The convergence guarantee applies to grounded, lower-triangular interaction graphs with a suitable solver; raising the maximum population cap beyond 8 buys only marginal improvement in exploitability; and the MuJoCo Football result is more a qualitative demonstration of feasibility than a systematic benchmark win. Even so, the through-line — carry an entire population in one conditional model — points a clear and economical way toward pushing game-theoretic population learning into genuinely complex games.